МИНИСТЕРСТВО СЕЛЬСКОГО ХОЗЯЙСТВА РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«РОССИЙСКИЙ ГОСУДАРСТВЕННЫЙ АГРАРНЫЙ УНИВЕРСИТЕТ – МСХА имени К. А. ТИМИРЯЗЕВА»

(ФГБОУ ВО РГАУ – МСХА имени К. А. Тимирязева)

Институт экономики и управления

Кафедра статистики и кибернетики

Направление: 09.03.02 Информационные системы и технологии

Направление:

Системный анализ

КУРСОВОЙ ПРОЕКТ

на тему: «Проектирование и разработка многопользовательской блог-платформы с интеллектуальным помощником»

Выполнила:
Студентка 3 курса группы Д-Э317
Преснякова Вероника Борисовна

Дата регистрации КП
на кафедре: _____

Допущен к защите
Руководитель:

_____
учёная степень, учёное звание, ФИО

Члены комиссии:

_____ _____
учёная степень, учёное звание, ФИО подпись

_____ _____
учёная степень, учёное звание, ФИО подпись

Оценка _____

Дата защиты _____

Москва 2026

МИНИСТЕРСТВО СЕЛЬСКОГО ХОЗЯЙСТВА РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«РОССИЙСКИЙ ГОСУДАРСТВЕННЫЙ АГРАРНЫЙ УНИВЕРСИТЕТ – МСХА имени К. А. ТИМИРЯЗЕВА»

Институт экономики и управления АПК

Кафедра статистики и кибернетики

ЗАДАНИЕ
НА КУРСОВОЙ ПРОЕКТ (КП)

Обучающаяся: Преснякова Вероника Борисовна

Тема КП: Методы и средства проектирования информационных систем и технологий

Исходные данные к работе:

Требования к веб-приложению (поддержка локальной работы, наличие элементов геймификации), стек веб-технологий (Node.js, React, PostgreSQL), теоретическая база по проектированию баз данных и паттернам архитектуры.

Перечень подлежащих разработке в работе вопросов:

1. Анализ предметной области и обоснование выбора программных средств.
2. Разработка математической модели игровой экономики.
3. Проектирование архитектуры приложения и реляционной схемы базы данных.
4. Разработка серверной части и клиентского интерфейса.
5. Тестирование системы и формирование плана масштабирования до многопользовательской блог-платформы с интеллектуальным помощником.

Перечень дополнительного материала:

Схема реляционной базы данных (ER-диаграмма), UML-диаграммы, листинги исходного кода приложения.

Дата выдачи задания: «___» _____ 20__ г.

Руководитель (подпись, ФИО): _____

Задание принял к исполнению (подпись обучающегося): _____

«___» _____ 20__ г.

РЕЦЕНЗИЯ

на курсовую работу/проект обучающегося

Федерального государственного бюджетного образовательного учреждения высшего образования
«Российский государственный аграрный университет – МСХА имени К. А. Тимирязева»

Обучающаяся: Преснякова Вероника Борисовна

Учебная дисциплина: «Методы и средства проектирования информационных систем и технологий»

Тема курсовой работы/проекта: «Проектирование и разработка многопользовательской блог-платформы с интеллектуальным помощником»

Полнота раскрытия темы:

_____

_____

_____

_____

_____

Оформление:

_____

_____

Замечания:

_____

_____

_____

_____

Курсовой проект отвечает предъявляемым к нему требованиям
и заслуживает _____ оценки.

(отличной, хорошей, удовлетворительной, неудовлетворительной)

Рецензент _____

(фамилия, имя, отчество, уч. степень, уч. звание, должность, место работы)

Дата: «___» _____ 20__ г. Подпись: _____

Аннотация

Тема исследования: Проектирование и разработка многопользовательской блог-платформы с интеллектуальным помощником.

Автор: Преснякова Вероника Борисовна, студентка группы Д-Э-317, РГАУ-МСХА им. К. А. Тимирязева.

Научный руководитель: Калитвин Владимир Анатольевич, доцент.

В ходе работы над курсовым проектом спроектирована базовая архитектура веб-приложения для управления контентом с элементами геймификации. По итогам проектирования реализована локальная система ведения записей и интегрирован модуль виртуального питомца, состояние которого динамически рассчитывается на основе активности пользователя. Инструментами реализации выступили платформа Node.js, база данных PostgreSQL и библиотека пользовательских интерфейсов React. В качестве дальнейших перспектив развития проекта запланировано масштабирование разработанной системы до многопользовательской блог-платформы, а также трансформация модуля виртуального питомца в полноценного интеллектуального помощника.

Ключевые слова: веб-приложение, управление контентом, геймификация, виртуальный питомец, блог-платформа, интеллектуальный помощник.

Содержание

Введение **6**

Глава 1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ И АРХИТЕКТУРНЫЕ ПАТТЕРНЫ ВЕБ-ПРИЛОЖЕНИЙ **7**

1.1. Анализ предметной области, аналогов и формулирование требований к системе 7

1.2. Паттерны чистой архитектуры и организация веб-приложений 10

1.3. Обоснование технологического стека 16

Глава 2. ПРОЕКТИРОВАНИЕ И ПРОГРАММНАЯ РЕАЛИЗАЦИЯ СИСТЕМЫ **20**

2.1. Математическая модель игровой экономики 20

2.2. Проектирование реляционной схемы базы данных 20

2.3. Механизм полнотекстового поиска на основе функциональных инвертированных индексов 25

2.4. Проектирование программного интерфейса и контрактная безопасность 27

2.5. Проектирование динамических сценариев взаимодействия компонентов 29

2.6. Программная реализация серверной и клиентской логики 31

2.7. Процесс развертывания и обеспечение безопасности . . . 33

2.8. Стратегия тестирования и контроль качества 35

Заключение **38**

Список использованных источников **39**

Введение

В условиях постоянного увеличения объемов цифровой информации и возрастающих требований к безопасности персональных данных актуализируется задача эффективного структурирования заметок и оперативных записей. Существующие программные решения зачастую характеризуются избыточной функциональной сложностью, создающей повышенную когнитивную нагрузку на пользователя, либо не содержат механизмов для поддержания регулярности ведения записей.

Актуальность темы обусловлена необходимостью создания автономного, локально развертываемого веб-приложения для управления заметками с игровой моделью мотивации. Минималистичная организация интерфейса позволяет снизить порог входа, а интеграция системы ухода за виртуальным питомцем стимулирует регулярность работы и самодисциплину пользователя.

Целью курсового проекта является проектирование и разработка веб-приложения заметок с игровой подсистемой, обладающего строгой сквозной типизацией и оптимизированной архитектурой хранения данных.

Для достижения поставленной цели необходимо спроектировать базовую архитектуру исходя из вариантов использования и требований предметной области, провести анализ и обосновать выбор технологий, разработать математическую модель игровой экономики приложения на основе многомерной взвешенной средней (расчет индекса удовольствия, шкалы здоровья питомца и системы коэффициентов опыта у пользователя), спроектировать схему базы данных, реализовать серверную часть, клиентский интерфейс с поддержкой оптимистичных обновлений, тестирование и локальное развертывание.

Глава 1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ И АРХИТЕКТУРНЫЕ ПАТТЕРНЫ ВЕБ-ПРИЛОЖЕНИЙ

1.1. Анализ предметной области, аналогов и формулирование требований к системе

В современных условиях лавинообразного роста объемов информации задачи персонального информационного менеджмента приобретают первостепенное значение. Эффективность работы специалиста напрямую зависит от скорости фиксации возникающих идей, структурирования задач и надежности доступа к накопленной базе знаний. Однако анализ пользовательского опыта взаимодействия с существующими программными средствами выявил фундаментальное противоречие: с одной стороны, усложнение функциональности приложений приводит к резкому росту когнитивной нагрузки и снижению регулярности их использования; с другой стороны, отсутствие элементов мотивации и обратной связи превращает базу заметок в пассивный массив неструктурированных данных («кладбище заметок»).

Дополнительным фактором является вопрос информационной безопасности и суверенности данных. Подавляющее большинство коммерческих платформ функционируют исключительно по облачной модели предоставления программного обеспечения как услуги, что сопряжено с рисками несанкционированного доступа третьих лиц, утечек конфиденциальной информации, а также внезапной блокировки учетных записей или сетевой недоступности удаленных серверов. В этой связи актуализируется разработка автономных систем с локальным размещением, сочетающих простоту использования, гарантированную конфиденциальность и механизмы поведенческого удержания пользователей.

Для определения функциональных границ и ключевых преимуществ разрабатываемой системы был проведен сравнительный анализ наиболее распространенных решений на рынке персональных систем управления информацией.

Сервис Google Keep представляет собой легковесное облачное решение для быстрого создания заметок, ключевыми достоинствами которого выступа-

ют минималистичный блочный интерфейс, высокая скорость создания записей и цветовая маркировка. К существенным недостаткам системы относятся жесткая привязка к экосистеме вендора, невозможность автономного развертывания на локальном сервере, ограниченные возможности структурирования связей тегов и отсутствие механизмов стимулирования регулярности ведения записей.

В свою очередь, платформа Notion предлагает комплексную рабочую среду на базе блочной модели данных и реляционных таблиц с высочайшей гибкостью настройки, однако чрезмерная функциональная перегруженность создает высокий порог входа, время начальной загрузки интерфейса оказывается значительным, а работа системы полностью зависит от постоянного интернет-соединения.

Локально-ориентированное приложение Obsidian обеспечивает хранение базы знаний в открытом виде на основе текстовых файлов Markdown и графа обратных ссылок, но требует от пользователя навыков самостоятельной конфигурации каталогов и плагинов, не предоставляя из коробки глубокого морфологического анализа русскоязычных словоформ.

Наконец, приложение Habitica демонстрирует высокую эффективность игровой мотивации за счет механики ролевой игры, однако ориентировано исключительно на дискретные списки задач и не приспособлено для структурированного хранения развернутых текстовых заметок.

Результаты сопоставительного анализа представлены в таблице 1.1.

Таблица 1.1. Сравнительный анализ систем персонального информационного менеджмента

Критерий сравнения	**Google Keep**	**Notion**	**Obsidian**	**Habitica**	**Разрабатываемая система**
Автономность на собственном сервере	Нет	Нет	Да	Частично	**Да (контейнеризация)**
Контроль и приватность данных	Низкая (облако)	Низкая (облако)	Высокая	Средняя	**Абсолютная (локальная база)**
Скорость создания заметки менее 3 сек	Высокая	Низкая	Средняя	Низкая	**Высокая (оптимистичный интерфейс)**
Полнотекстовый поиск с морфологией	Ограниченный	Ограниченный	Базовый	Отсутствует	**Высокий (инвертированный индекс)**
Организация связей заметок и меток	Метки	Реляционные базы	Граф ссылок	Теги задач	**Связь многие-ко-многим (третья нормальная форма)**
Игровая модель удержания пользователя	Отсутствует	Отсутствует	Отсутствует	Высокая (ролевая игра)	**Встроенная (индекс питомца)**
Нагрузка на ресурсы клиента	Минимальная	Высокая	Средняя	Средняя	**Минимальная (компактный код)**

На основе проведенного анализа сформулирован комплекс функциональных требований к проектируемому веб-приложению. Система должна обеспечивать создание, чтение, редактирование и мягкое удаление текстовых заметок с помещением их в корзину, возможность скрытия записей в архив без физического уничтожения связанных метаданных, а также категоризацию на основе связи «многие ко многим» с произвольным количеством пользовательских меток. Наряду с этим критически важными функциональными требованиями выступают полнотекстовый поиск по заголовкам и содержимому заметок с поддержкой русской морфологии и ранжированием релевантности, а также регистрация активности пользователя в журнале событий с периодическим пересчетом характеристик виртуального питомца.

В части нефункциональных требований к системе предъявляются строгие критерии производительности, отзывчивости и безопасности. Время ответа сервера на операции чтения и поиска заметок не должно превышать 50 миллисекунд при объеме базы данных до ста тысяч записей. Для нивелирования задержек сети пользовательский интерфейс должен строиться на базе оптимистичных обновлений состояния. Структура хранения данных обязана строго соответствовать правилам третьей нормальной формы с каскадным контролем внешних ключей, а серверные процессы должны функционировать в изолированных контейнерах от имени непривилегированного пользователя операционной системы с обязательной валидацией входных потоков данных на границе приложения.

1.2. Паттерны чистой архитектуры и организация веб-приложений

В начале создания веб-приложения требуется проектирование архитектуры как логики работы, так и пользовательского интерфейса. Архитектура приложений должна отражать конкретные бизнес-процессы, а именно варианты использования системы. Главной задачей чистой архитектуры является организация структуры программного обеспечения таким образом, чтобы модули системы непосредственно отражали ее предметную область и оставались независимыми от внешних технологических фреймворков и библиотек [2].

В разрабатываемом веб-приложении серверная часть программы опиралась на главный функционал: создание, редактирование, архивация и удаление заметок с их организацией с помощью тегов. В начале разработки в части интерфейса архитектура не требовалась, поскольку это один из способов вывода пользователю данных и архитектура должна являться нейтральной к представлению вариантов использования.

Для систематизации функциональных требований и снижения связности компонентов общая модель вариантов использования системы была декомпозирована на три независимых функциональных модуля. Первый модуль заметок (рисунок 1.1) охватывает базовые операции создания, получения списка, полнотекстового поиска, фильтрации по статусам и детального просмотра заметок. Второй модуль корзины и управления тегами (рисунок 1.2) регламентирует сценарии мягкого удаления, двухэтапной очистки корзины, восстановления документов и ассоциации заметок с пользовательскими метками. Третий модуль

справочника тегов и сервисов базы данных (рисунок 1.3) отвечает за операции над глобальным справочником меток, сквозную проверку валидации и взаимодействие с реляционной СУБД.

На рисунках 1.1, 1.2 и 1.3 представлены декомпозированные диаграммы вариантов использования:

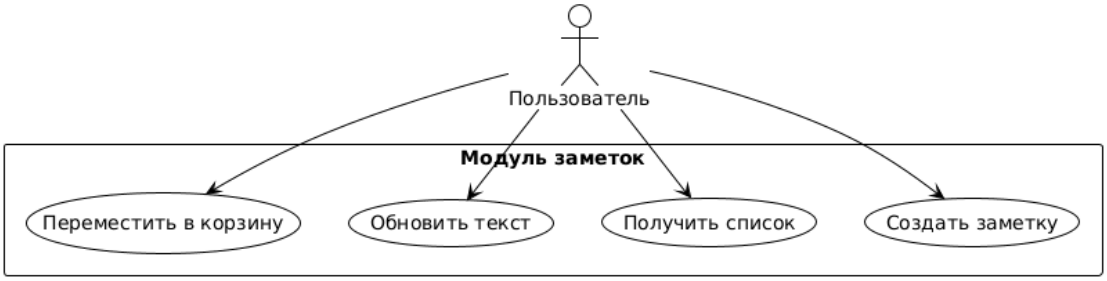


Рисунок 1.1 – Диаграмма вариантов использования: модуль заметок

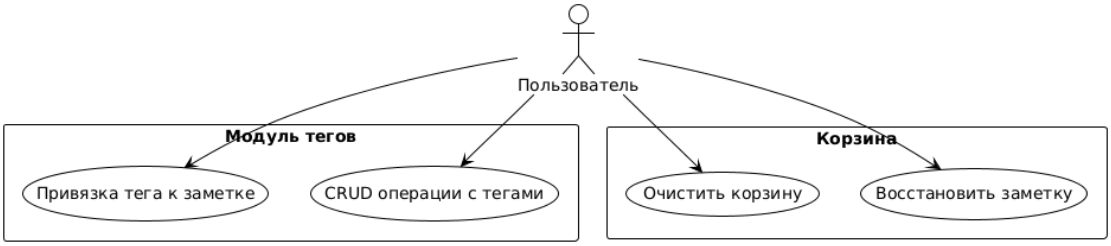


Рисунок 1.2 – Диаграмма вариантов использования: корзина и управление тегами

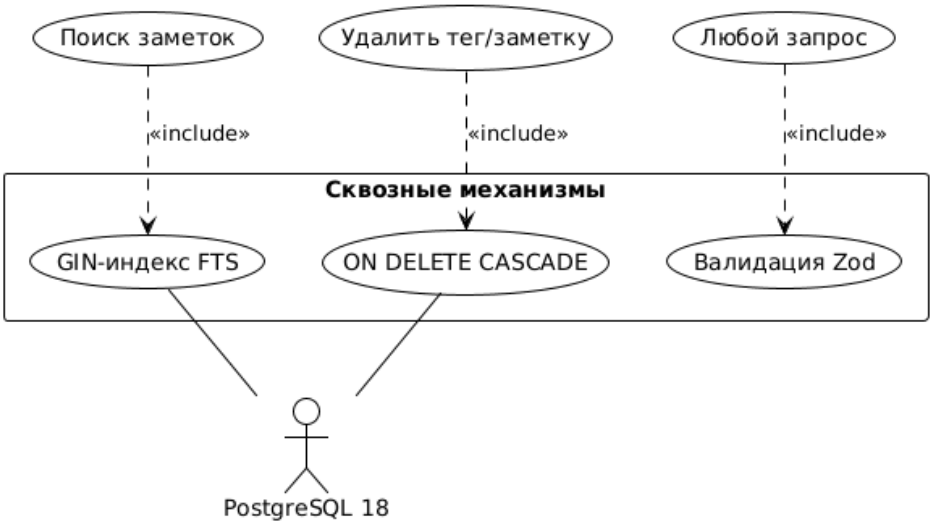


Рисунок 1.3 – Диаграмма вариантов использования: модуль тегов и сервисы базы данных

Существуют множество разных архитектур, и всех их объединяет деление задач на уровни: уровня бизнес-правил, интерфейсного и системного уровней. Каждый уровень требует зависимости от другого уровня, начиная от ядра (абстрактных правил, определяющих суть приложения) до периферии (компонентов пользовательского интерфейса, SQL-запросов, сетевых маршрутизаторов). При разработке возникают проблемы с масштабированием ПО, если зависимости неправильно направлены и компоненты зависят друг от друга. Таким образом, для зависимостей системы нужны правила и притом направленные в сторону ядра архитектуры, то есть верхний уровень не знает о нижнем [2].

В основе архитектуры серверной части веб-приложения лежит feature-based подход для изоляции компонентов системы в зависимости от её роли. Он состоит из цепочки роутера, контроллера и репозитория, где контроллер изолирован от сетевой инфраструктуры роутера и работает только с абстрактным интерфейсом репозитория для соблюдения принципа инверсии зависимостей. Этот способ организации кода позволяет легче тестировать, заменять компоненты и не зависеть от любых внешних уровней.

Помимо этого, в приложении используется паттерн «Репозиторий», который действует как прослойка между бизнес-логикой и уровнем работы с данными, предоставляя интерфейсы, напоминающие коллекции объектов [6]. Он скрывает от неё сложность SQL-запросов и специфику конкретной СУБД. Такой подход облегчает чтение кода и упрощает тестирование: благодаря инверсии зависимостей реальный репозиторий, работающий с PostgreSQL, можно заменить в модульных тестах на mock-объект. Это позволяет проверять бизнес-логику в полной изоляции от внешнего ввода-вывода [2].

В соответствии с методологией C4 архитектура серверной части декомпозирована на три ключевые подсистемы уровня компонентов. Модуль обработки заметок (рисунок 1.4) инкапсулирует цепочку маршрутизатора, валидационных схем Zod, контроллера и репозитория с полнотекстовым поиском. Модуль обработки тегов (рисунок 1.5) изолирует сценарии каталогизации, связывание сущностей через связующую таблицу и операции справочника меток. Сквозной слой ядра и обработки ошибок (рисунок 1.6) реализует централизованную валидацию входных схем, глобальный перехватчик сбоев с маппингом статус-кодов и управление пулом соединений базы данных.

На рисунках 1.4, 1.5 и 1.6 представлены архитектурные диаграммы компонентов бэкенда:

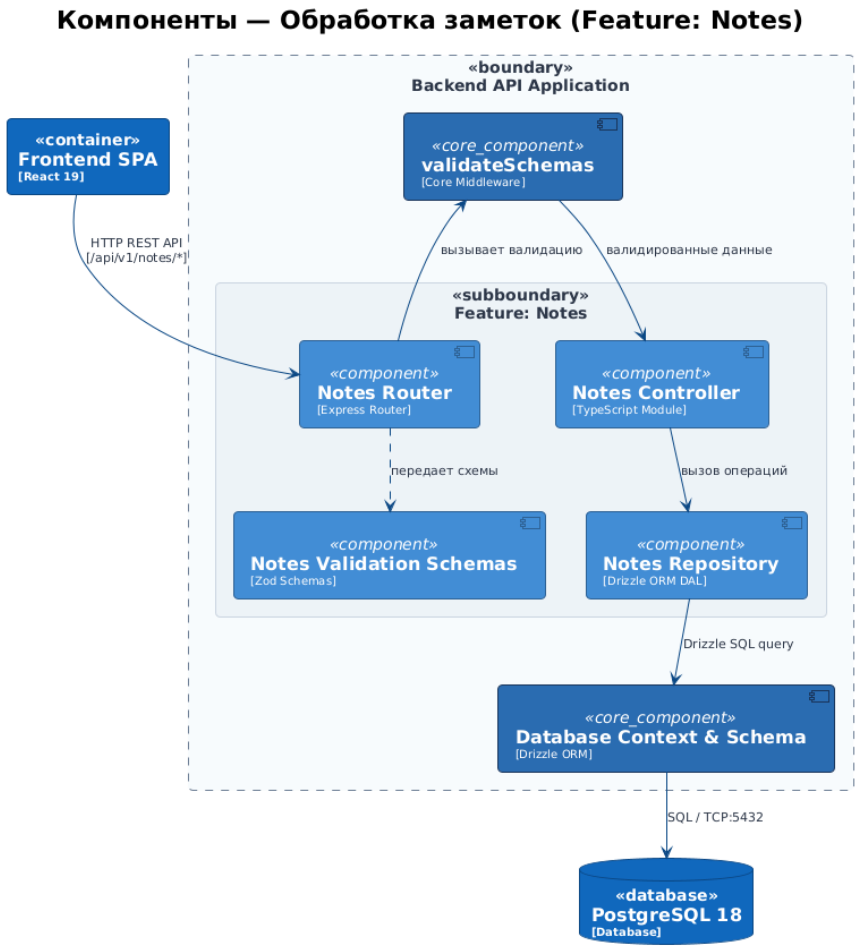


Рисунок 1.4 – Диаграмма компонента: обработка заметок

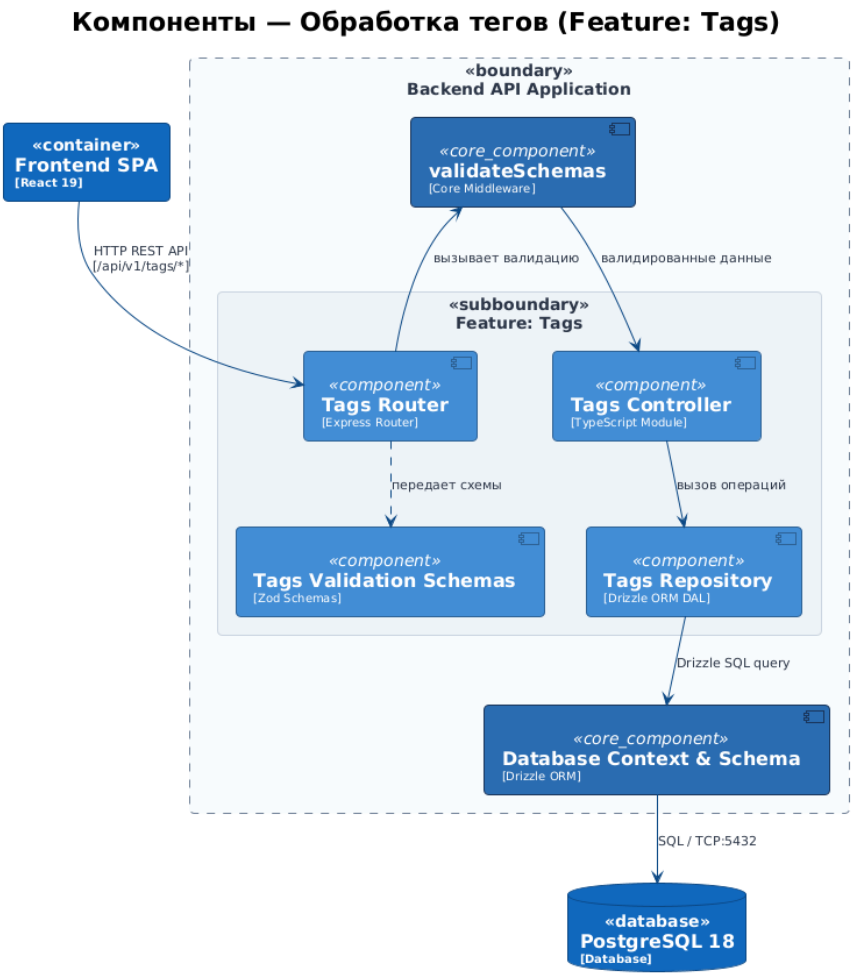


Рисунок 1.5 – Диаграмма компонента: обработка тегов

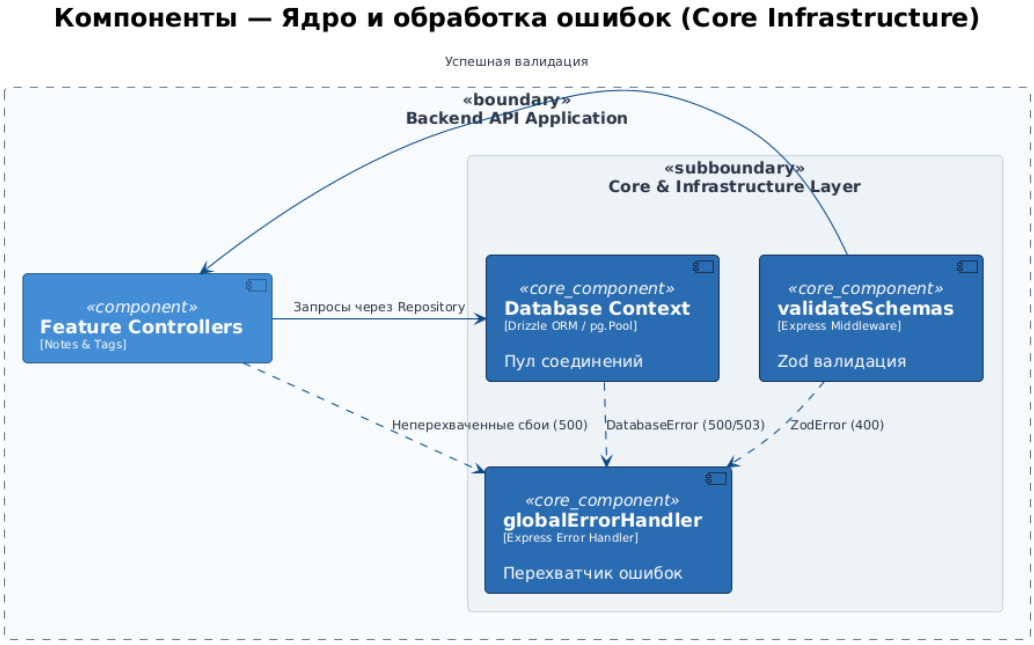


Рисунок 1.6 – Диаграмма ядра и обработки ошибок

Для клиентской части приложения архитектура должна отражать логику предметной области, но классическая группировка файлов по техническому типу (по каталогам компонентов, хуков и сервисов) становится неэффективной по мере роста кодовой базы и приводит к возникновению циклических зависимостей. Поэтому была выбрана архитектурная методология Feature-Sliced Design [7].

Архитектурная методология Feature-Sliced Design задает строгую иерархию слоев, направленную от глобальных модулей к локальным. На базовом слое App располагаются параметры инициализации приложения, корневые стили, провайдеры контекста и конфигурация маршрутизации. Вышестоящий слой Pages формирует полноценные пользовательские экраны путем композиции сущностей, фич и виджетов. Слой Widgets объединяет крупные самостоятельные композиционные блоки интерфейса, такие как панель навигации или дашборд питомца. Слой Features инкапсулирует сценарии взаимодействия, несущие непосредственную бизнес-ценность для пользователя, в то время как нижележащий слой Entities хранит бизнес-модели предметной области и логику их представления. Завершает иерархию инфраструктурный слой Shared, включающий переиспользуемые интерфейсные компоненты, утилиты и конфигурации сетевых клиентов, независимые от специфики предметной области.

1.3. Обоснование технологического стека

В качестве системы хранения и управления данными выбрана PostgreSQL, в отличие от документо-ориентированной MongoDB или синхронной SQLite. Выбор был основан на том, что данная СУБД позволяет выполнять операции чтения и записи параллельно без блокировок (многоверсионное управление конкурентным доступом) [9], а также бизнес-логика приложения (заметки, теги, связи между ними) несет реляционный характер. PostgreSQL также предоставляет развитый инструментарий полнотекстового поиска на уровне ядра СУБД, что имеет решающее значение для эффективной организации поиска по текстовым заметкам.

Для работы с СУБД был выбран инструмент Drizzle ORM [13]. В экосистеме TypeScript инструмент Prisma длительное время занимал лидирующие позиции, однако архитектурные особенности данного проекта делают выбор Drizzle ORM более предпочтительным. В основе работы Prisma лежит подход schema-first: структура базы данных декларируется в специальном файле `.prisma`, после чего запускается генерация строго типизированного клиента. Такая абстракция удобна при разработке, но использование ресурсоемкого движка выполнения приводит к повышенному потреблению оперативной памяти и увеличенным задержкам. В отличие от Prisma, Drizzle ORM строится на парадигме codebase-first и SQL-first, где схема БД описывается непосредственно на чистом TypeScript. Отказ от промежуточной кодогенерации позволяет выводить типы «на лету» еще на этапе написания кода. Ключевым достоинством Drizzle ORM являются сведенные к минимуму накладные расходы во время исполнения: библиотеки напрямую преобразуют вызовы функций в оптимизированный SQL-код без дополнительных слоёв логики, снижая время формирования запросов до значений порядка 0,5 мс.

В листинге 1.1 представлен фрагмент описания сущностей `pets`, `notes` и `tags` с использованием синтаксиса Drizzle ORM.

```
export const pets = pgTable('pets', {
  id: uuid('id')
    .primaryKey()
    .default(sql`gen_random_uuid()`),
  name: varchar('name', { length: 255 }).notNull(),
  level: integer('level').default(1).notNull(),
  progress: integer('progress').default(0).notNull(),
  pleasureIndex: integer('pleasure_index').default(100).notNull(),
})

export const notesTable = pgTable('notes', {
  id: uuid('id')
    .primaryKey()
    .default(sql`gen_random_uuid()`),
  user_id: uuid()
    .default(sql`gen_random_uuid()`)
    .notNull(),
  title: text().default('').notNull(),
  content: text().default('').notNull(),
  isArchive: boolean().default(false),
  isDeleted: boolean().default(false),
})

export const tagsTable = pgTable('tags', {
  id: uuid('id')
    .primaryKey()
    .default(sql`gen_random_uuid()`),
  name: text().notNull(),
})
```

Листинг 1.1: Описание схемы сущностей pets, notes и tags в Drizzle ORM

Серверная инфраструктура основывается на языке TypeScript, среде выполнения Node.js и веб-фреймворке Express 5 [1]. Выбор Node.js и TypeScript обосновывается унифицированием технологий на клиентской и серверной частях, позволяя не проводить миграцию серверной кодовой базы TypeScript на другие языки, такие как Java или Python. Использование TypeScript на всем стеке позволяет применять такие инструменты, как tRPC, для сквозной безопасности типов без кодогенерации, что сводит к минимуму переключение контекста.

TypeScript является надмножеством языка JavaScript с дополнительным синтаксисом определения типов. Благодаря модулю проверки в TypeScript код программы не запустится и не скомпилируется в JavaScript, пока в аннотации типов или синтаксисе будут ошибки. TypeScript, будучи статическим, моделирует процессы выполнения JavaScript, в отличие от языка Python со стро-

гой типизацией. Например, сложить число и строку все еще будет валидно в TypeScript из-за приведения типов к строке. Если попытаться проделать ту же операцию в Python и других языках, то это незамедлительно вызовет ошибку выполнения.

Хотя TypeScript имеет проверку типов, его нельзя назвать надежным, так как в таком случае система типов должна гарантировать точность статических типов. В сравнении с архитектурными задачами Python, который соблюдает строгость типов непосредственно во время выполнения, TypeScript обеспечивает безопасность только при написании и сборке кода, оставляя стандартный JavaScript [19].

Среды для TypeScript, такие как Node.js, используют однопоточный событийный цикл, который показывает эффективность при обработке тысяч одновременных сетевых соединений без блокировки основного потока. В то же время Python ограничен глобальной блокировкой интерпретатора, из-за чего он уступает в пропускной способности при выполнении задач ввода-вывода даже при использовании асинхронных фреймворков.

При этом сферы эффективного применения языков программирования разграничены: Python занимает устойчивые лидирующие позиции в областях машинного обучения, математического моделирования и аналитической обработки данных, что обусловлено наличием развитой экосистемы библиотек с оптимизированными расширениями на C/C++. В TypeScript же нет сопоставимых по уровню оптимизации аналогов, поэтому прагматичнее для тяжелых аналитических вычислений писать сервисы на Python.

На клиентской стороне приложения архитектура состояния разделена на две независимые области. Фреймворк React 19 в связке со сборщиком Vite обеспечивает компонентную модель пользовательского интерфейса и высокую скорость горячей перезагрузки модулей в процессе разработки. Для управления серверным состоянием — данными, которые асинхронно извлекаются из API и требуют кэширования, дедупликации запросов и фоновой синхронизации — внедрена библиотека TanStack Query v5 [18].

Управление локальным пользовательским состоянием, таким как состояние модальных окон, тема оформления или временные фильтры, делегировано библиотеке Zustand. В отличие от React Context или Redux, Zustand предоставляет минималистичный API, основанный на хуках, и точечно обновляет ком-

поненты, предотвращая избыточный рендеринг всего дерева документа веб-приложения.

Сквозная валидация данных реализуется с помощью библиотеки Zod, позволяющей единожды описать схему данных и использовать ее как для валидации на сервере, так и для форм на клиенте. Визуальная составляющая формируется посредством Tailwind CSS v4, оптимизирующего объем результирующего CSS-кода за счет применения атомарных служебных классов.

Глава 2. ПРОЕКТИРОВАНИЕ И ПРОГРАММНАЯ РЕАЛИЗАЦИЯ СИСТЕМЫ

2.1. Математическая модель игровой экономики

Ключевой особенностью платформы, отличающей ее от аналогичных решений, выступает сложная математическая модель игровой экономики. Главной задачей данной архитектуры является формирование устойчивого паттерна возврата пользователей и обеспечение их долгосрочного удержания за счет заботы о виртуальном питомце.

Центральной метрикой системы выступает индекс удовольствия. Вычисление данного показателя базируется на алгоритме многомерной средней. Модель не ограничивается простой агрегацией количества написанных символов, а оценивает комплексную пользу пользовательских действий через систему весовых коэффициентов. В расчет принимаются такие векторы, как регулярность сессий, объем и структурность создаваемых заметок, а также частота использования тегов для организации информации.

Полученный индекс удовольствия выполняет функцию динамического множителя при накоплении опыта, что в дальнейшем позволяет виртуальному питомцу визуально расти и повышать свой уровень. В случае поддержания высокого значения индекса, что отражает активное ведение базы заметок, к получаемому базовому опыту применяется стимулирующий коэффициент 1,2. При длительном бездействии алгоритм фиксирует падение метрики, активируя штрафной множитель 0,5. Для защиты системы от аномалий использования и предотвращения генерации избыточных фиктивных записей внедрено ограничение: максимальный прирост опыта не может превышать 500 единиц за одни календарные сутки.

2.2. Проектирование реляционной схемы базы данных

Проектирование реляционной схемы базы данных реализовано в строгом соответствии с требованиями третьей нормальной формы. Данное архитектурное решение исключает дублирование информации и защищает систему от аномалий модификации. Архитектура хранилища разделена на две функциональ-

ные подсистемы: модуль управления и категоризации заметок и аналитическое ядро геймификации с журналом активности питомца.

На рисунке 2.1 представлена ER-диаграмма первой части реляционной модели данных, отражающая структуру сущностей заметок, тегов и промежуточной таблицы отношений между ними.



Рисунок 2.1 – ER-диаграмма реляционной базы данных: модуль заметок и тегов

Первая часть архитектуры представляет собой детальную подсистему для управления текстовым контентом и его категоризации, спроектированную с учетом высоких требований к производительности поиска и гибкости хранения. Центральным узлом выступает таблица заметок, которая содержит пользовательский текст, заголовок и служебные метаданные, необходимые для контроля жизненного цикла каждой записи. Флаги архивации и мягкого удаления позволяют скрывать информацию от пользователя без ее физического уничтожения из базы данных, а автоматические временные метки создания и обновления обеспечивают правильную хронологическую сортировку. Для обеспечения высокого быстродействия поиска даже при существенных объемах текстовой информации предусмотрен специализированный GIN-индекс, преобразующий контент в векторный формат лексем. Механизм категоризации

декомпозирован: теги хранятся в независимом справочнике с ограничением уникальности наименований, что исключает возникновение смысловых дубликатов. Связь между заметками и тегами организуется через промежуточную таблицу, реализующую отношение «многие ко многим». Данное решение позволяет ассоциировать множество меток с отдельным документом и гарантирует ссылочную целостность: за счет механизма каскадного удаления при окончательном удалении заметки база данных автоматически ликвидирует связанные внешние ключи.

В листинге 2.1 представлена программная реализация связующей таблицы и отношений многие-ко-многим с использованием Drizzle ORM.

```
export const noteTagsTable = pgTable(
  'noteTags',
  {
    note_id: uuid('note_id')
      .notNull()
      .references(() => notesTable.id, { onDelete: 'cascade' }),
    tag_id: uuid('tag_id')
      .notNull()
      .references(() => tagsTable.id, { onDelete: 'cascade' }),
  },
  (table) => [primaryKey({ columns: [table.tag_id, table.note_id] })]
)

export const noteTagsRelations = defineRelations({ tagsTable, notesTable,
    noteTagsTable }, (r) => ({
  tagsTable: {
    notesTable: r.many.notesTable({
      from: r.tagsTable.id.through(r.noteTagsTable.tag_id),
      to: r.notesTable.id.through(r.noteTagsTable.note_id),
    }),
  },
  notesTable: {
    tagsTable: r.many.tagsTable({
      from: r.notesTable.id.through(r.noteTagsTable.note_id),
      to: r.tagsTable.id.through(r.noteTagsTable.tag_id),
    }),
  },
}))
```

Листинг 2.1: Реализация связи многие-ко-многим между заметками и тегами в Drizzle ORM

На рисунке 2.2 представлена ER-диаграмма аналитического ядра геймификации, фиксирующая взаимосвязь сущностей питомца, матрицы уровней и журнала активности.

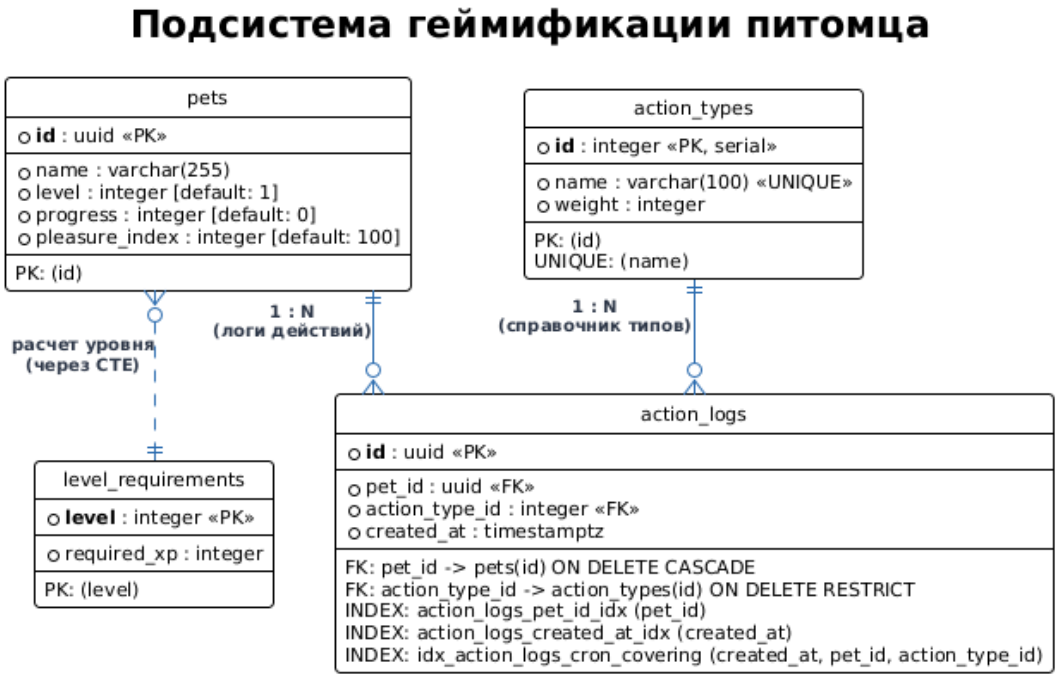


Рисунок 2.2 – ER-диаграмма подсистемы геймификации и журнала активности питомца

Вторая часть схемы описывает полноценное аналитическое ядро геймификации, построенное на принципах строгой нормализации данных, что защищает игровую механику от логических ошибок и обеспечивает точный аудит всей активности. Главная таблица профилей хранит моментальный снимок текущего состояния виртуального питомца: его идентификаторы, накопленный числовой прогресс, текущий уровень и индекс настроения, который выступает индикатором регулярности ухода. Чтобы алгоритм понимал, в какой момент питомец должен эволюционировать, введена отдельная матрица требований к уровням, а именно компактная справочная таблица, задающая пороги опыта для перехода на каждую следующую ступень развития. Сама же механика начисления характеристик основана на непрерывном учете событий через таблицу логов, которая фиксирует каждое взаимодействие с привязкой к точному времени. Допустимые типы взаимодействий вынесены в настраиваемый справочник, где каждому действию присвоен свой весовой коэффициент, определяющий ценность поступка. Такая событийная архитектура позволяет перезапи-

сывать статус питомца и хранить всю историческую хронологию его развития. Продуманная система составных индексов в таблице логов гарантирует, что фоновые процессы и агрегирующие запросы будут моментально рассчитывать аналитику за любой период, не создавая нагрузку на сервер даже при миллионах накопленных действий.

В листинге 2.2 приведено описание таблицы журнала действий `action_logs` и ее индексов.

```
export const actionLogs = pgTable(
  'action_logs',
  {
    id: uuid('id')
      .primaryKey()
      .default(sql`gen_random_uuid()`),
    petId: uuid('pet_id')
      .notNull()
      .references(() => pets.id, { onDelete: 'cascade' }),
    actionTypeId: integer('action_type_id')
      .notNull()
      .references(() => actionTypes.id, { onDelete: 'restrict' }),
    createdAt: timestamp('created_at', { withTimezone: true })
      .defaultNow()
      .notNull(),
  },
  (table) => [
    index('action_logs_pet_id_idx').on(table.petId),
    index('action_logs_created_at_idx').on(table.createdAt),
    index('idx_action_logs_cron_covering').on(
      table.createdAt,
      table.petId,
      table.actionTypeId
    ),
  ]
)
```

Листинг 2.2: Описание таблицы журнала событий action_logs и индексов

Производительность фоновых задач критически зависит от индексов. В базе данных спроектированы составные покрывающие индексы. Они сконструированы таким образом, что содержат не только ключи для фильтрации по полям идентификатора пользователя и времени создания, но и сами агрегируемые значения. Это позволяет планировщику PostgreSQL выполнять запросы в режиме сканирования только индексов. В этом режиме данные извлекаются исключительно из структуры индекса, расположенной в оперативной памяти, без

необходимости ресурсоемкого чтения основных страниц таблицы с диска, что обеспечивает масштабируемость системы [8].

2.3. Механизм полнотекстового поиска на основе функциональных инвертированных индексов

Поиск информации в массиве текстовых заметок является одной из наиболее часто выполняемых операций. В традиционных веб-приложениях начального уровня поиск нередко реализуется с помощью операторов сопоставления шаблонов строк. Однако данное решение обладает неустранимыми архитектурными дефектами. Прежде всего, использование шаблонов поиска по подстроке принципиально исключает возможность применения стандартных индексных структур B-Tree, вынуждая СУБД выполнять полное последовательное чтение всех страниц таблицы с диска, что приводит к резкому падению быстродействия по мере накопления данных. Кроме того, при таком подходе полностью отсутствует лингвистический анализ, поскольку посимвольное сопоставление не учитывает грамматику естественного языка, в результате чего поиск по заданной форме слова не позволяет обнаружить записи с иными грамматическими формами той же основы.

Для преодоления указанных ограничений в системе реализован встроенный механизм полнотекстового поиска СУБД PostgreSQL. Преобразование текстовой информации осуществляется в рамках последовательных двух этапов. На этапе токенизации и синтаксического разбора исходный текстовый поток разделяется на элементарные составляющие, включая слова, числовые последовательности и служебные разделители. Далее на этапе нормализации специализированный словарь для русской конфигурации отсекает окончания и суффиксы, отфильтровывает шумовые лексические единицы (предлоги, союзы и междометия), преобразуя оставшиеся слова в нормализованные лексемы.

Для формирования поискового документа применяется функция `to_tsvector`, которая преобразует конкатенацию заголовка и содержимого заметки в вектор сортированных лексем с указанием их позиций. Обработка пользовательской строки поиска выполняется функцией `websearch_to_tsquery`. Данная функция формирует корректное логическое дерево запроса без риска синтаксических сбоев, поддерживая фразовый

поиск в кавычках, исключение слов через знак минуса и неявное логическое «И».

Вычисление вектора лексем во время выполнения каждого пользовательского запроса создавало бы чрезмерную нагрузку на процессорные мощности сервера базы данных. Классическим решением является создание отдельного столбца для хранения предвычисленного вектора, однако это влечет за собой увеличение расхода дисковой памяти.

В разработанной системе применен более эффективный подход: функциональный обобщенный инвертированный индекс GIN. Индекс формируется непосредственно по определенному выражению объединения текстовых полей:

```
CREATE INDEX "search_index" ON "notes" USING gin (
  to_tsvector('russian', coalesce("title", '') || ' ' || coalesce("content", ''))
);
```

Листинг 2.3: Создание функционального GIN-индекса для полнотекстового поиска

В листинге 2.4 представлен фрагмент реализации поискового запроса в репозитории заметок с использованием Drizzle ORM.

```
if (search) {
  conditions.push(
    sql`to_tsvector('russian', coalesce(${notesTable.title}, '') || ' ' || coalesce(
    ${notesTable.content}, '')) @@ websearch_to_tsquery('russian', ${search})`
  )
}
const query = db
  .select(noteSelectFields)
  .from(notesTable)
  .where(conditions.length > 1 ? and(...conditions) : conditions[0])
  .orderBy(desc(notesTable.updatedAt))
  .limit(10);
```

Листинг 2.4: Реализация полнотекстового поиска в репозитории заметок

Для верификации эффективности спроектированного индекса было проведено сравнительное профилирование запросов с помощью встроенного инструмента анализа планов выполнения `EXPLAIN ANALYZE` на синтетическом наборе данных объемом 50 000 заметок.

При выполнении поискового запроса без использования специализированного индекса планировщик базы данных применил метод полного последовательного сканирования таблицы без задействования индексов. В процессе

обработки запроса СУБД прочитала 3 420 страниц буферной памяти, а суммарное время выполнения составило 148.62 мс.

Напротив, при задействовании разработанного функционального инвертированного индекса GIN планировщик выполнил эффективное битовое сканирование индекса `search_index` с последующей точечной выборкой страниц таблицы. При этом объем прочитанных данных сократился до 8 страниц индекса и 14 страниц таблицы, а время отклика снизилось до 0.84 мс. Таким образом, внедрение инвертированного индекса обеспечило ускорение поисковой операции более чем в 175 раз при минимальной нагрузке на подсистему дискового ввода-вывода.

2.4. Проектирование программного интерфейса и контрактная безопасность

Взаимодействие между клиентской частью приложения и серверным ядром строится в строгом соответствии с архитектурным стилем REST. Сетевой интерфейс организован вокруг двух ключевых доменных сущностей системы: модуля управления заметками (включающего сценарии создания, модификации, фильтрации, архивации и мягкого удаления) и справочника тегов (обеспечивающего независимое управление метками и их ассоциацию с документами).

При разработке распределенных клиент-серверных систем на платформе TypeScript распространенной проблемой является рассогласование контрактов данных. Ручное дублирование типов запросов, параметров маршрутизации и структур ответов на стороне клиента и сервера неизбежно приводит к трудноуловимым ошибкам при модернизации программного интерфейса. Для устранения данного риска в проекте применен подход на основе единого источника истины .

В качестве фундамента контракта выступают декларативные схемы валидации библиотеки Zod, расширенные специализированным модулем преобразования в спецификацию OpenAPI. Такой подход позволяет описывать валидационные правила для входных данных (заголовки, длина текста, корректность идентификаторов) один раз непосредственно в кодовой базе сервера. На основе этих же правил во время сборки автоматически генерируется машиночитаемый документ спецификации OpenAPI версии 3.1.

В листинге 2.5 представлен фрагмент регистрации схем и путей API с использованием модуля `@asteasolutions/zod-to-openapi`.

```
export const NoteSchema = registry.register(
  'Note',
  z.object({
    id: z.string().uuid(),
    title: z.string().max(255),
    content: z.string().max(1000),
    isArchive: z.boolean().nullable().optional(),
    isDeleted: z.boolean().nullable().optional(),
    createdAt: z.string().datetime(),
    updatedAt: z.string().datetime(),
  })
)

registry.registerPath({
  method: 'post',
  path: '/api/v1/notes',
  request: {
    body: {
      content: { 'application/json': { schema: NoteCreateInSchema } }
    }
  },
  responses: {
    201: { description: 'Заметка успешно создана', content: { 'application/json': {
    schema: NoteSchema } } },
    400: defaultErrorResponse,
  }
})
```

Листинг 2.5: Декларация контракта API и схем валидации через Zod-to-OpenAPI

На стороне клиентского приложения данный документ спецификации используется библиотекой `openapi-fetch` для автоматического вывода статических типов сетевого уровня. В отличие от традиционных сетевых библиотек, требующих ручной типизации каждого запроса через обобщенные интерфейсы, разработанный клиент автоматически сопоставляет запрашиваемый путь, обязательные параметры строки запроса и структуру передаваемого тела. При попытке обращения к несуществующему маршруту или передаче некорректного набора свойств компилятор TypeScript блокирует сборку проекта до этапа развертывания. Таким образом достигается сквозная типобезопасность всей системы от схемы валидации на сервере до компонентов пользовательского интерфейса.

2.5. Проектирование динамических сценариев взаимодействия компонентов

Архитектурная надежность системы определяется не только статической структурой модулей, но и динамикой их взаимодействия в условиях асинхронных операций и потенциальных сбоев сети. В рамках проекта детально спроектированы два критически важных сценария: обработка изменения данных с оптимистичным обновлением интерфейса и регламентный ночной пересчет игровых характеристик.

Для исключения видимой задержки интерфейса при взаимодействии пользователя с системой реализован паттерн оптимистичных мутаций средствами библиотеки TanStack Query v5 [18]. Традиционная модель отправки данных требует ожидания полного сетевого цикла «клиент–сервер–клиент», что при задержках мобильных или нестабильных сетей вызывает визуальное зависание формы.

Разработанный цикл обработки пользовательской мутации организован как непрерывный детерминированный процесс синхронизации состояния. В момент отправки формы обработчик мутации активирует фазу предварительной подготовки `onMutate`, на которой менеджер запросов немедленно останавливает фоновые обновления списка заметок через метод `cancelQueries`, предотвращая перезапись оптимистичного состояния устаревшими ответами.

Следом выполняется глубокое копирование текущего состояния локального кэша с помощью `getQueryData` для формирования снимка отката, после чего в кэш синхронно помещается временная запись с локально сгенерированным идентификатором, обеспечивая немедленную перерисовку пользовательского интерфейса без задержки.

Параллельно в фоновом режиме отправляется асинхронный сетевой запрос к серверному эндпоинту, где полезная нагрузка валидируется промежуточным слоем Zod и передается в репозиторий для сохранения в базе данных PostgreSQL.

В случае сетевого обрыва или возникновения ошибки валидации срабатывает фаза обработки сбоя `onError`: система восстанавливает структуру локального кэша из ранее сохраненного снимка отката и выводит диагностическое предупреждение в интерфейсе.

При получении успешного ответа сервера на фазе `onSuccess` временная запись в кэше замещается персистентным объектом, содержащим постоянный идентификатор UUID и системные временные метки, после чего выполняется инвалидация связанных ключей запросов.

Второй динамический сценарий обеспечивает периодическую агрегацию активности пользователей и пересчет характеристик питомца. Критическим требованием к фоновым задачам является предотвращение состояния гонки , когда при параллельном запуске нескольких экземпляров сервера один и тот же суточный журнал событий обрабатывается повторно, приводя к неконтролируемому начислению опыта.

Для решения проблемы конкурентности применен механизм транзакционных рекомендательных блокировок базы данных: функция `pg_try_advisory_xact_lock` [15, 16, 17]. В отличие от стандартных строковых блокировок таблицы, рекомендательные блокировки не создают накладных расходов на дисковые структуры и управляются исключительно в оперативной памяти диспетчера блокировок СУБД.

Алгоритм регламентного пересчета функционирует в строгом транзакционном контексте. При наступлении назначенного времени рабочий сервис открывает транзакцию в СУБД и инициирует неблокирующий захват целочисленного идентификатора с помощью указанной функции рекомендательной блокировки `pg_try_advisory_xact_lock(42)`.

Если параллельный процесс уже удерживает данный ключ, функция немедленно возвращает отказ, после чего текущая транзакция безопасно завершается откатом `ROLLBACK` без ожидания и без блокировки потока выполнения.

В случае успешного получения блокировки сервис отправляет оптимизированный пакетный SQL-запрос с обобщенным табличным выражением CTE, который агрегирует суммарный заработанный опыт по суточным журналам событий и атомарно обновляет уровень и текущий прогресс в таблице состояния питомцев.

По завершении транзакции посредством команды `COMMIT` рекомендательная блокировка автоматически освобождается ядром PostgreSQL, исключая возникновение дедлоков и зависших мьютексов.

2.6. Программная реализация серверной и клиентской логики

Практическая реализация серверной логики фокусируется на обеспечении отказоустойчивости и многоуровневой защите. Централизованная валидация данных реализована с использованием промежуточного слоя обработки. Любой входящий HTTP-запрос (например, создание заметки) перехватывается промежуточным слоем, где его полезная нагрузка сопоставляется со строгими схемами, описанными на базе Zod. Если данные не соответствуют контракту (например, отсутствует заголовок или текст превышает допустимый лимит), промежуточный слой прерывает дальнейшую обработку и возвращает клиенту статус 400. Это гарантирует, что в контроллеры и репозитории поступают исключительно верифицированные данные, предотвращая инъекции и сбои в бизнес-логике [1].

Одной из самых сложных алгоритмических задач в бэкенде является реализация функции `processDailyPetProgress`. Поскольку расчет дневного прогресса зависит от сотен записей в таблице `action_logs`, построчное извлечение и обновление каждой записи на уровне Node.js неизбежно привело бы к проблеме множественных запросов и переполнению пула соединений. Для оптимизации этой задачи используются обобщенные табличные выражения, реализуемые через оператор WITH в SQL. В листинге 2.6 представлен расчет нового уровня и агрегация опыта с помощью CTE.

```
const recalculationQuery = sql<{
  id: string
  old_level: number
  new_level: number
  new_progress: number
}>`
  WITH daily_xp AS (
    SELECT
      ${actionLogs.petId} AS pet_id,
      SUM(${actionTypes.weight})::int AS earned_xp
    FROM ${actionLogs}
    INNER JOIN ${actionTypes}
      ON ${actionLogs.actionTypeId} = ${actionTypes.id}
    WHERE
      ${actionLogs.createdAt} >= ${startDate}
      AND ${actionLogs.createdAt} < ${endDate}
    GROUP BY ${actionLogs.petId}
  )
  UPDATE ${pets} AS p
  SET
    level = CASE
      WHEN lr.required_xp IS NOT NULL
           AND (p.progress + dxp.earned_xp) >= lr.required_xp
        THEN p.level + 1
      ELSE p.level
    END,
    progress = CASE
      WHEN lr.required_xp IS NOT NULL
           AND (p.progress + dxp.earned_xp) >= lr.required_xp
        THEN (p.progress + dxp.earned_xp) - lr.required_xp
      ELSE p.progress + dxp.earned_xp
    END
  FROM daily_xp dxp
  LEFT JOIN ${levelRequirements} lr
    ON lr.level = p.level
  WHERE p.id = dxp.pet_id
  RETURNING
    p.id,
    (p.level - CASE
      WHEN lr.required_xp IS NOT NULL AND (p.progress + dxp.earned_xp) >= lr.
    required_xp THEN 1
      ELSE 0
    END) AS old_level,
    p.level AS new_level,
    p.progress AS new_progress;
`
```

Листинг 2.6: Агрегация опыта и пересчет уровня питомца с использованием CTE

Интерфейс пользователя спроектирован с учетом высоких требований к отзывчивости пользовательского опыта. Для нивелирования сетевых задержек реализован паттерн оптимистичных обновлений средствами TanStack Query v5 [18]. При добавлении тега или заметки интерфейс не дожидается ответа от сервера. В функции `onMutate` система делает снимки текущего состояния кэша, после чего производит локальное обновление данных, обеспечивая немедленную перерисовку интерфейса без ожидания ответа сервера [14]. Фоном запрос отправляется на сервер. Если сервер возвращает ошибку (например, потеря соединения), срабатывает механизм отката в блоке `onError`, который восстанавливает состояние интерфейса до ранее сохраненного снимка. Контроль форм и их реактивность осуществляются библиотекой `react-hook-form`, которая бесшовно интегрирована с Zod-схемами [1].

На рисунке 2.3 представлен интерфейс главного экрана веб-приложения, объединяющий сетку заметок и панель виртуального питомца.

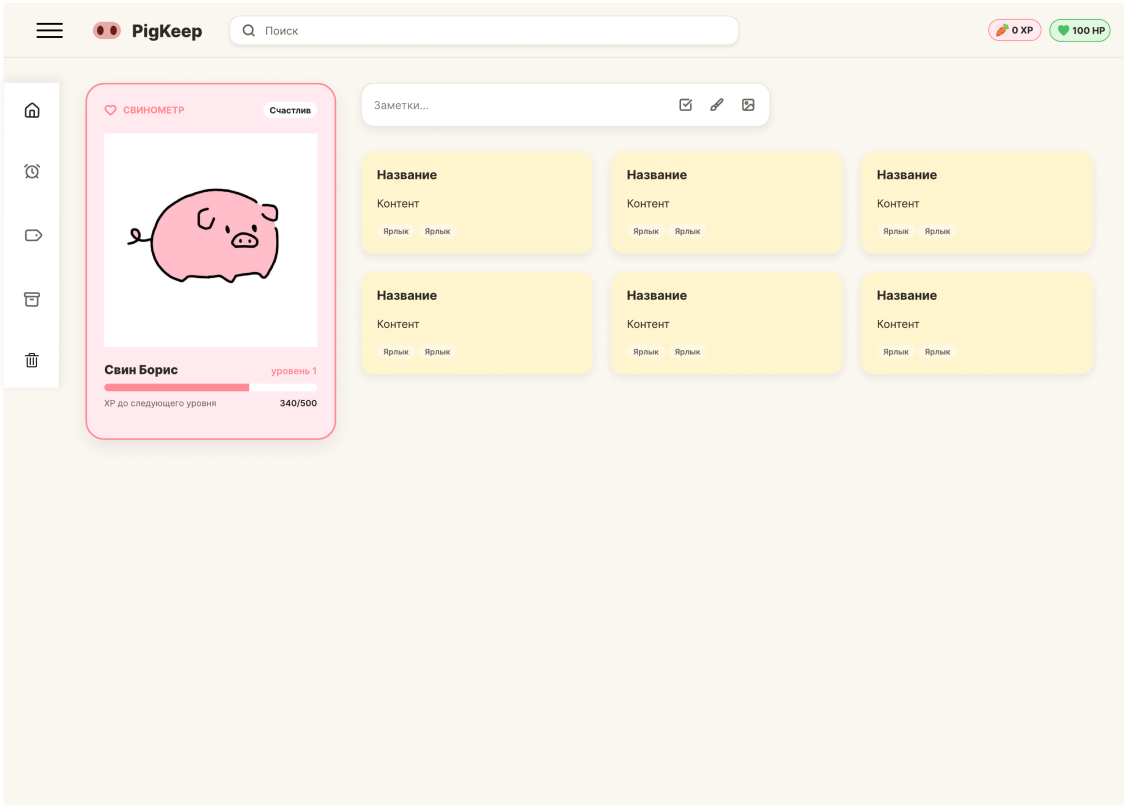


Рисунок 2.3 – Интерфейс главного экрана веб-приложения

2.7. Процесс развертывания и обеспечение безопасности

Процесс развертывания системы стандартизирован посредством контейнеризации в Docker с применением многоэтапных сборок. Этот подход решает

две задачи: минимизацию размера образа и повышение безопасности. На первом этапе в контейнер устанавливаются все зависимости, выполняется проверка типов TypeScript и сборка React-приложения через Vite. На втором этапе создается чистый, легковесный образ на базе Nginx. В него копируются только скомпилированные артефакты из папки dist. Важнейшим аспектом безопасности является использование образа `nginx-unprivileged`, который запускает веб-сервер от имени ограниченного системного пользователя без прав суперпользователя. Благодаря такому подходу итоговый размер Docker-образа фронтенда составляет 26.5 МБ [3].

В листинге 2.7 приведен многоэтапный Dockerfile для сборки и развертывания серверной части приложения.

```
FROM node:24.21.0-bookworm-slim AS builder
WORKDIR /usr/src/app
COPY package*.json ./
COPY backend/package*.json ./backend/
COPY packages/shared/package*.json ./packages/shared/
RUN npm ci
COPY packages/shared ./packages/shared
COPY backend ./backend
WORKDIR /usr/src/app/backend
RUN npm run build:prod

FROM node:24.21.0-bookworm-slim AS runner
WORKDIR /usr/src/app
COPY package*.json ./
COPY backend/package*.json ./backend/
COPY packages/shared/package*.json ./packages/shared/
RUN npm ci --omit=dev
COPY --from=builder /usr/src/app/packages/shared ./packages/shared
COPY --from=builder /usr/src/app/backend/build ./backend/build
COPY --from=builder /usr/src/app/backend/migrations ./migrations
EXPOSE 3000
USER node
CMD [ "node", "backend/build/server.js"]
```

Листинг 2.7: Многоэтапный Dockerfile для серверной части приложения

В листинге 2.8 приведен Dockerfile для развертывания клиентской части с сервером Nginx от имени непривилегированного пользователя.

```
FROM node:24.21.0-bookworm-slim AS builder
WORKDIR /usr/src/app
COPY package*.json ./
COPY frontend/package*.json ./frontend/
COPY packages/shared/package*.json ./packages/shared/
RUN npm ci
COPY packages/shared ./packages/shared
COPY frontend ./frontend
WORKDIR /usr/src/app/frontend
RUN npm run build

FROM nginxinc/nginx-unprivileged:alpine AS runner
COPY frontend/nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=builder /usr/src/app/frontend/dist /usr/share/nginx/html
EXPOSE 8080
USER nginx
CMD ["nginx", "-g", "daemon off;"]
```

Листинг 2.8: Многоэтапный Dockerfile для развертывания клиентской части

2.8. Стратегия тестирования и контроль качества

Обеспечение качества программного обеспечения базируется на комплексном тестировании. Модульное тестирование алгоритмов и схем валидации выполняется в среде Vitest, которая обладает нативной поддержкой конфигураций Vite. Для проверки интеграции пользовательского интерфейса и логики запросов без развертывания реальной базы данных применяется библиотека Mock Service Worker. Она перехватывает сетевые запросы на уровне сервис-воркера в браузере или Node.js, позволяя возвращать заранее подготовленные мок-ответы. Это дает возможность правильно тестировать, гарантированно вызывать ошибки сервера для проверки корректности работы механизма оптимистичного отката в интерфейсе [14].

В проекте реализована комплексная пирамида тестирования, охватывающая как клиентскую, так и серверную части, с фокусом на проверке реального поведения системы. На уровне контрактов с помощью Vitest и Zod проверяются граничные условия без запуска сервера. Интеграционное тестирование серверных модулей с использованием Supertest, Drizzle ORM и PostgreSQL проверяет сквозное прохождение HTTP-запросов, работу полнотекстового поиска и соответствие БД нормальным формам.

В листинге 2.9 представлено тестирование эндпоинтов с помощью библиотеки Supertest и тестовой базы данных.

```
describe('GET /api/v1/pets', () => {
  it('should auto-initialize and return default pet when no pet exists', async () =>
    {
    const res = await request(app).get(BASE_URL)

    expect(res.status).toBe(200)
    expect(res.body.pet).toBeDefined()
    expect(res.body.pet).toMatchObject({
      name: 'Борис',
      level: 1,
      progress: 0,
      pleasureIndex: 100,
      requiredXp: 500,
      mood: 'happy',
    })
    expect(res.body.pet.id).toBeDefined()
  })
})
```

Листинг 2.9: Интеграционный тест эндпоинта API питомца с использованием Supertest

В листинге 2.10 представлено тестирование схем валидации и соответствия колонок базы данных требованиям 3NF.

```
describe('Database Schemas (3NF)', () => {
  it('should define pets table with correct columns', () => {
    expect(pets.id).toBeDefined()
    expect(pets.name).toBeDefined()
    expect(pets.level).toBeDefined()
    expect(pets.progress).toBeDefined()
    expect(pets.pleasureIndex).toBeDefined()
  })
})
```

Листинг 2.10: Тестирование колонок схемы базы данных на соответствие 3NF

На фронтенде интеграционное тестирование оценивает реальные пользовательские сценарии, состояния загрузки и обработку ошибок интерфейса. Юнит-тестирование хуков гарантирует корректную инвалидацию кэша и управление таймерами, что предотвращает утечки памяти при размонтировании компонентов. В листинге 2.11 представлено тестирование компонента `PetCard` с использованием React Testing Library.

```
describe('PetCard', () => {
  const mockPet: Pet = {
    id: 'pet-123',
    name: 'Борис',
    level: 2,
    progress: 340,
    requiredXp: 500,
    pleasureIndex: 90,
    mood: 'happy',
  }

  it('should render pet name with Свин"" prefix, level, and XP progress', () => {
    renderWithProviders(<PetCard pet={mockPet} />)

    expect(screen.getAllByText('Свин Борис').length).toBeGreaterThan(0)
    expect(screen.getAllByText('уровень 2').length).toBeGreaterThan(0)
    expect(screen.getByText('340/500')).toBeInTheDocument()
    expect(screen.getByText('XP до следующего уровня')).toBeInTheDocument()
  })
})
```

Листинг 2.11: Тестирование компонента PetCard с использованием React Testing Library

Для лучшей читаемости кода каждый тест строится по принципу Arrange-Act-Assert и помимо этого используются вспомогательные функции, исключая условия и дублирование кода. Изоляция тестов достигается за счет полной очистки выделенной БД перед каждым запуском, применения независимых клиентов для кэша и оборачивания компонентов в кастомные провайдеры контекста (листинг 2.12).

```
beforeEach(async () => {
  await test_db.execute(sql`TRUNCATE TABLE ${notesTable} CASCADE`)
})
```

Листинг 2.12: Изоляция тестовых наборов через очистку таблиц базы данных

Заключение

В ходе выполнения курсового проекта было установлено, что надежность и масштабируемость веб-приложения закладываются на начальном этапе архитектурного проектирования. Разделение ответственности между слоями как на сервере, так и в интерфейсе требует дополнительных трудозатрат на этапе конфигурации, однако обеспечивает отсутствие циклических зависимостей и предсказуемость поведения кодовой базы. При интеграции механизмов игровой мотивации с базовым функционалом управления заметками модульная изоляция гарантирует сохранение стабильности системы за счет четкого разграничения задач компонентов.

Проектирование и реализация серверной части и схемы базы данных подтвердили значимость строгой сквозной типизации данных. Синхронизация валидации входящих запросов, моделей базы данных и состояния пользовательского интерфейса в рамках единого контракта позволяет устранять ошибки на этапе статической компиляции. Кроме того, выполнение расчетов прогресса и агрегации связанных данных средствами реляционной СУБД с использованием обобщенных табличных выражений обеспечивает атомарность операций и снижает нагрузку на вычислительные ресурсы сервера.

На этапе разработки клиентской части было показано, что субъективное восприятие производительности веб-приложения определяется не только физической задержкой сети, но и эффективностью управления локальным состоянием. Реализация паттерна оптимистичных обновлений с механизмом безопасного отката изменений обеспечивает минимальное время отклика интерфейса и непрерывность пользовательского взаимодействия. Таким образом, в ходе работы спроектирована и реализована целостная клиент-серверная система, обеспечивающая согласованное взаимодействие компонентов от уровня хранения данных до элементов пользовательского интерфейса.

Список использованных источников

[1] Кашьяро, М. Шаблоны проектирования Node.js / М. Кашьяро, Л. Маммино. — Санкт-Петербург : Питер, 2022. — 832 с. — Текст : непосредственный. (дата обращения: 05.10.2026).

[2] Мартин, Р. Чистая архитектура. Искусство разработки программного обеспечения / Р. Мартин. — Санкт-Петербург : Питер, 2021. — 352 с. — Текст : непосредственный. (дата обращения: 05.10.2026).

[3] Architectural Patterns Revisited – A Pattern Language : веб-сайт. — URL: https://eprints.cs.univie.ac.at/2698/1/ArchPatterns.pdf (дата обращения: 05.10.2026).

[4] Does Domain-Driven Design Lead to Finding the Optimal Modularity : веб-сайт. — URL: http://ieeexplore.ieee.org/iel7/6287639/9312710/09359794.pdf (дата обращения: 05.10.2026).

[5] An Approach for Mapping Domain-Specific AOM Applications to a : веб-сайт. — URL: https://www.jucs.org/jucs_20_4/an_approach_for_mapping/jucs_20_04_0534_0560_matsumoto.pdf (дата обращения: 05.10.2026).

[6] A Pattern Language for Metadata-based Frameworks. — Текст : электронный. — URL: https://bia.unibz.it/view/pdfCoverPage?instCode=39UBZ_INST&filePid=13300047980001241&download=true (дата обращения: 05.10.2026).

[7] Feature Sliced Design : [сайт]. — Текст : электронный. — URL: https://feature-sliced.design/ (дата обращения: 05.10.2026).

[8] Postgres Concurrency Internals: Connections, Locks, and MVCC : веб-сайт. — URL: https://abhimanyunagurkar.com/blog/postgres-concurrency-internals/ (дата обращения: 05.10.2026).

[9] MVCC in PostgreSQL: Benefits and Challenges : веб-сайт. — URL: https://www.scribd.com/document/835805809/Multi-Version-C

oncurrency-Control-MVCC-in-PostgreSQL (дата обращения: 05.10.2026).

[10] Multiversion Concurrency Control (MVCC) in PostgreSQL : веб-сайт. — URL: https://www.geeksforgeeks.org/postgresql/multiversion-concurrency-control-mvcc-in-postgresql/ (дата обращения: 05.10.2026).

[11] Concurrency Challenges in Database Systems: A Focus on Postgresql : веб-сайт. — URL: https://www.researchgate.net/publication/391365712_Concurrency_Challenges_in_Database_Systems_A_Focus_on_Postgresql (дата обращения: 05.10.2026).

[12] Prisma vs Drizzle: Rich ORM or Lightweight Builder? : веб-сайт. — URL: https://www.13labs.au/compare/prisma-vs-drizzle (дата обращения: 05.10.2026).

[13] Drizzle vs Prisma: Choosing the Right TypeScript ORM : веб-сайт. — URL: https://betterstack.com/community/guides/scaling-nodejs/drizzle-vs-prisma/ (дата обращения: 05.10.2026).

[14] Optimistic UI & Mutations : веб-сайт. — URL: https://fearchitect.com/topics/optimistic-ui-mutations (дата обращения: 05.10.2026).

[15] PostgreSQL Advisory Locks with pg_advisory_lock : веб-сайт. — URL: https://www.postgresscripts.com/post/postgresql-advisory-locks-with-pg-advisory-lock/ (дата обращения: 05.10.2026).

[16] How to Use Advisory Locks in PostgreSQL : веб-сайт. — URL: https://www.cybrosys.com/research-and-development/postgres/how-to-use-advisory-locks-in-postgresql (дата обращения: 05.10.2026).

[17] Solving concurrency issue with postgresql advisory locks : веб-сайт. — URL: https://www.mandricmihai.com/2021/01/solving-concurrency-issue-with-advisory-locks-in-python.html (дата обращения: 05.10.2026).

[18] Optimistic Updates | TanStack Query React Docs : веб-сайт. — URL: https://tanstack.com/query/latest/docs/framework/react/guides/optimistic-updates (дата обращения: 05.10.2026).

[19] Вандеркам, Д. Эффективный TypeScript. 62 способа улучшить код / Д. Вандеркам. — Санкт-Петербург : Питер, 2020. — 288 с. — Текст : непосредственный. (дата обращения: 05.10.2026).